CSC10014 / COMPUTATIONAL THINKING / 2026

ENGINEERING A SMART VIRTUAL ASSISTANT

PROJECT AT A GLANCE

TEAM	DURATION	CORE OUTPUT	EVIDENCE
5–7 students	**10-week project arc**	**Working intelligent system**	**Design + code + evaluation**

What this project is really about

The goal is not to maximize model complexity. The goal is to turn an ambiguous human need into a reliable computational process: understand → decompose → represent → decide → execute → verify → improve.

01 / CHALLENGE & LEARNING GOALS

A modern computing project framed around judgment, system design, and evidence — not just code generation.

Why this project?

In academic and professional environments, users rarely express problems as clean specifications. They ask vague questions, provide incomplete constraints, mix factual lookup with planning, and expect the system to know when it should search, calculate, ask for clarification, or refuse. A Smart Virtual Assistant is therefore an ideal Computational Thinking project: the technical challenge is not merely generating text, but designing the computational process behind the response.

By the end of the project, your team should be able to:

- Transform an ill-defined real-world need into a well-scoped and testable computational problem.
- Decompose a complex assistant into explicit capabilities, modules, data flows, and interfaces.
- Choose appropriately among rules, algorithms, databases, retrieval, machine learning, LLMs, and external APIs.
- Design a system with explicit routing, validation, fallback, and failure handling rather than a single “send everything to the LLM” path.
- Evaluate behavior using repeatable test cases, metrics, error analysis, and at least one documented improvement cycle.
- Communicate technical decisions through diagrams, pseudocode, code, evidence, and an engineering defense.

02 / WHAT YOU WILL BUILD

A coherent intelligent system that converts human requests into computational actions.

Your team will design and implement a Smart Virtual Assistant for a clearly defined user group and problem context. The assistant may support learning, productivity, document work, university life, software development, or another instructor-approved domain.

A strong system should follow an explicit pipeline:

REQUEST	UNDERSTAND	ROUTE	RETRIEVE / COMPUTE	REASON	VERIFY	RESPOND / ACT

Design principle

AI is one computational component, not the architecture. Use AI where uncertainty, language, perception, or synthesis justifies it. Use deterministic logic where correctness, constraints, or repeatability matter more.

03 / MINIMUM SYSTEM REQUIREMENTS

These requirements define the expected engineering depth; teams may go beyond them when justified.

ID	Requirement	What good evidence looks like
R1	**Problem boundary**	A specific target user, pain point, 8–12 representative use cases, explicit non-goals, constraints, and measurable success criteria.
R2	**Deterministic capability**	At least one rule/algorithm/calculation/workflow that is transparent and testable.
R3	**Data / retrieval capability**	At least one structured or unstructured information source with documented provenance, metadata, and retrieval logic.
R4	**AI-enabled capability**	At least one AI/ML/LLM capability whose use is technically justified by the problem.
R5	**Routing / orchestration**	An explicit mechanism that decides which capability should handle a request; include clarification and fallback paths.
R6	**End-to-end interface**	A user-facing interface or demonstrable API flow connected to the real backend/system logic.
R7	**Reliability & safeguards**	Input validation, error handling, timeout/failure behavior, privacy rules, and relevant misuse or prompt-injection defenses.
R8	**Evaluation**	A repeatable benchmark/test set, metrics or evaluation rubric, error taxonomy, and evidence of at least one improvement cycle.

04 / SUGGESTED PROJECT TRACKS

Choose a domain because it creates an interesting computational problem — not because it sounds fashionable.

A	**Intelligent Study Companion** Turn deadlines, progress, learning resources, and available time into an adaptive study plan. Core ideas: Scheduling • ranking • retrieval • learner modeling
B	**Personal Productivity Assistant** Convert unstructured requests into tasks, priorities, constraints, and executable schedules. Core ideas: Task decomposition • priority queues • constraint solving
C	**Intelligent Document Workspace** Search, compare, and synthesize PDFs, notes, reports, and structured records with traceable evidence. Core ideas: Parsing • indexing • retrieval • RAG • provenance
D	**Meeting Intelligence Assistant** Extract decisions, action items, owners, deadlines, and unresolved issues across meetings. Core ideas: Information extraction • temporal reasoning • state tracking
E	**University Life Assistant** Combine course, campus, regulation, and schedule information into a student-facing assistant. Core ideas: Search • structured data • planning • routing
F	**Software Development Assistant** Support requirement breakdown, code/document retrieval, test generation, debugging, and change summaries. Core ideas: Dependency analysis • program context • testing • retrieval

05 / REFERENCE ARCHITECTURE

A reference, not a mandatory stack. Your architecture should be simpler when the problem allows it.

USER / CLIENT	Chat or form UI • CLI • mobile/web prototype
↓	
INPUT UNDERSTANDING	Intent • entities • constraints • validation
↓	
ORCHESTRATION	Routing • state • policy • tool selection • fallback
↓	
CAPABILITY LAYER	Algorithms / rules | retrieval / search | AI / ML / LLM
↓	
DATA & TOOLS	Database • documents • APIs • calendar • approved external services
↓	
VALIDATION & SAFETY	Evidence checks • constraint checks • privacy • guardrails • escalation
↓	
OBSERVABILITY & EVALUATION	Logs • feedback • test cases • metrics • error analysis

06 / SUBMISSION PACKAGE

Treat the submission as an engineering handoff: another team should be able to understand, run, test, and critique your system.

01 Engineering Report

12–18 pages recommended, excluding appendices. Problem framing, CT reasoning, architecture, data/knowledge design, algorithms/routing, AI role, implementation, risks, evaluation, errors, limitations, roadmap.

02 Source Repository / Demo

Source code, environment/dependency files, sample configuration, reproducible setup/run instructions, and a working demo. Never submit real credentials.

03 Architecture & Design Pack

System-context diagram, component diagram, sequence/data-flow diagram, key schemas/data models, pseudocode, and important interface contracts.

04 Evaluation Pack

Benchmark/test set, expected outputs/labels where applicable, evaluation script or rubric, metrics, error taxonomy, regression tests, before/after evidence.

05 Failure & Risk Pack

Failure catalogue, threat/privacy analysis, red-team cases, safeguards, fallback/escalation paths, and unresolved risks.

06 Presentation & Final Demo

Concise technical story: problem → key decisions → architecture → evidence → demo → limitations → lessons learned.

07 Team Logbook & Contribution Evidence

Weekly task allocation, issues/PRs, meeting/progress notes, contribution summary, and a short record of how AI tools were used and verified.

07 / EVALUATION RUBRIC

The rubric rewards evidence of thinking, system quality, and disciplined iteration — not code volume or model size.

Criterion	Weight	What distinguishes strong work
Problem framing & CT reasoning	**15%**	Clear user/problem boundary; measurable success; strong decomposition, abstraction, and explicit trade-offs.
System architecture & orchestration	**15%**	Coherent components, interfaces, data/control flow, routing, fallback, and failure-aware design.
Data / retrieval layer	**10%**	Appropriate sources, metadata, provenance, retrieval logic, and traceability.
Algorithm & technology selection	**10%**	Well-justified choice of deterministic logic, search, AI, tools, and APIs; avoids unnecessary complexity.
MVP & integration	**20%**	End-to-end system works reliably; components are actually integrated; code/repository is understandable and reproducible.
Reliability, security & responsible AI	**10%**	Failure model, privacy/safety thinking, safeguards, red-team cases, and appropriate abstention/escalation.
Testing & evaluation	**10%**	Balanced benchmark, meaningful metrics/rubric, error analysis, regression testing, and before/after improvement.
Report, demo & presentation	**10%**	Professional technical communication, evidence-centered demo, clear ownership, and ability to defend decisions.

Assessment note

Final quality matters, but weekly milestone evidence also matters. A polished Week-8 demo cannot fully compensate for weak problem framing, undocumented decisions, fabricated evaluation, or an inability to explain how the system works.

08 / POLICIES, TECHNICAL EXPECTATIONS & RESOURCES

Use modern AI tools as engineering aids, while preserving understanding, accountability, and reproducibility.

Team & workflow expectations

- Team size: 5–7 students unless approved otherwise.
- Maintain a shared Git repository, issue/task board, and weekly project log.
- Every member should contribute across analysis, design, implementation, evaluation, and communication—not only one isolated role.
- Important design changes should be documented briefly (for example in README notes, issues, or lightweight ADR-style records).

AI tool policy

- ChatGPT, Claude, Gemini, Copilot, and similar tools may be used for brainstorming, code assistance, debugging, writing support, or test generation.
- The team remains responsible for understanding, adapting, verifying, and testing all generated content or code.
- AI-generated text/code is not evidence by itself. Technical claims and evaluation results must come from the team's sources, code, measurements, or experiments.
- Do not upload secrets, private credentials, restricted data, or sensitive university/company information to public repositories or external AI services.